

Design Patterns: A Complete Guide

A structured and beginner-friendly reference to the three major categories of software design patterns — Creational, Structural, and Behavioral.

Design patterns are reusable solutions to commonly occurring problems in software design. They are not code — they are blueprints or templates that guide how to structure your code to solve a problem efficiently and elegantly.

Table of Contents

- [1. Creational Design Patterns](#)
 - [Singleton](#)
 - [Factory Method](#)
 - [Abstract Factory](#)
 - [Builder](#)
 - [Prototype](#)
 - [2. Structural Design Patterns](#)
 - [Adapter](#)
 - [Bridge](#)
 - [Composite](#)
 - [Decorator](#)
 - [Facade](#)
 - [Flyweight](#)
 - [Proxy](#)
 - [3. Behavioral Design Patterns](#)
 - [Chain of Responsibility](#)
 - [Command](#)
 - [Iterator](#)
 - [Mediator](#)
 - [Memento](#)
 - [Observer](#)
 - [State](#)
 - [Strategy](#)
 - [Template Method](#)
 - [Visitor](#)
-

1. Creational Design Patterns

Overview

Creational patterns deal with **how objects are created**. They abstract the instantiation process, making a system independent of how its objects are created, composed, and represented. These patterns give you more flexibility in deciding which objects need to be created for a given situation.

Singleton

Description: Ensures that a class has only **one instance** throughout the lifetime of an application and provides a global access point to that instance.

- **Why it is used:** To control access to shared resources such as a configuration manager, logger, or database connection — where having multiple instances would cause conflicts or waste resources.
 - **Where it is commonly used:** Logging systems, application configuration managers, thread pools, caching systems.
-

Code Example (JAVA)

```
class Database {
    private static Database instance;

    private Database() {
    }

    public static Database getInstance() {
        if (instance == null) {
            instance = new Database();
        }
        return instance;
    }

    public void query(String sql) {
        System.out.println("Executing: " + sql);
    }
}

public class singleton {
    public static void main(String[] args) {
        Database db1 = Database.getInstance();
        Database db2 = Database.getInstance();
        System.out.println("Are db1 and db2 the same instance? " + (db1 ==
db2));
    }
}
```

Factory Method

Description: Defines an interface for creating an object, but **lets subclasses decide** which class to instantiate. The parent class defers object creation to its child classes.

- **Why it is used:** To promote loose coupling by eliminating the need to bind application-specific classes into your code. The creation logic is centralized and easy to extend.
 - **Where it is commonly used:** UI frameworks (creating different buttons for different OS), plugin systems, document generators.
-

Code Example (JAVA)

```
interface Transport {
    void deliver();
}

class Truck implements Transport {
    public void deliver() {
        System.out.println("Deliver by land in a box.");
    }
}

class Ship implements Transport {
    public void deliver() {
        System.out.println("Deliver by sea in a container.");
    }
}

abstract class Logistics {
    public void planDelivery() {
        Transport transport = createTransport();
        transport.deliver();
    }

    public abstract Transport createTransport();
}

class RoadLogistics extends Logistics {
    public Transport createTransport() {
        return new Truck();
    }
}

class SeaLogistics extends Logistics {
    public Transport createTransport() {
        return new Ship();
    }
}

public class factory_method {
    public static void main(String[] args) {
        Logistics logistics = new RoadLogistics();
        logistics.planDelivery();
    }
}
```

Abstract Factory

Description: Provides an interface for creating **families of related or dependent objects** without specifying their concrete classes.

- **Why it is used:** To ensure that a set of related objects are used together without mixing incompatible combinations. It's like a factory of factories.
 - **Where it is commonly used:** Cross-platform UI toolkits (e.g., Windows vs. macOS components), database drivers, theme systems.
-

Code Example (JAVA)

```
public class abstract_factory {
    interface Button { void render(); }
    interface Checkbox { void render(); }

    static class WinButton implements Button { public void render() {
System.out.println("Windows Button"); } }
    static class MacButton implements Button { public void render() {
System.out.println("Mac Button"); } }

    static class WinCheckbox implements Checkbox { public void render() {
System.out.println("Windows Checkbox"); } }
    static class MacCheckbox implements Checkbox { public void render() {
System.out.println("Mac Checkbox"); } }

    interface GUIFactory {
        Button createButton();
        Checkbox createCheckbox();
    }

    static class WinFactory implements GUIFactory {
        public Button createButton() { return new WinButton(); }
        public Checkbox createCheckbox() { return new WinCheckbox(); }
    }

    static class MacFactory implements GUIFactory {
        public Button createButton() { return new MacButton(); }
        public Checkbox createCheckbox() { return new MacCheckbox(); }
    }

    public static void main(String[] args) {
        GUIFactory factory;
        String os = "win";
        if (os.equals("win"))
            factory = new WinFactory();
        else
            factory = new MacFactory();

        Button button = factory.createButton();
        Checkbox checkbox = factory.createCheckbox();
        button.render();
        checkbox.render();
    }
}
```

Builder

Description: Separates the **construction of a complex object** from its representation, allowing the same construction process to create different representations.

- **Why it is used:** To construct objects step-by-step, especially when an object requires many configuration options or when different representations of a product are needed.
 - **Where it is commonly used:** Building complex query objects, constructing HTML/XML documents, configuring network requests.
-

Code Example (JAVA)

```
class Car {
    private String seats;
    private String engine;

    public void setSeats(String seats) {
        this.seats = seats;
    }

    public void setEngine(String engine) {
        this.engine = engine;
    }

    public void show() {
        System.out.println("Car with " + seats + " and " + engine);
    }
}

interface Builder {
    void reset();

    void setSeats(String seats);

    void setEngine(String engine);
}

class CarBuilder implements Builder {
    private Car car;

    public CarBuilder() {
        this.reset();
    }

    public void reset() {
        this.car = new Car();
    }

    public void setSeats(String seats) {
        this.car.setSeats(seats);
    }
}
```

```
        public void setEngine(String engine) {
            this.car.setEngine(engine);
        }

        public Car getProduct() {
            return this.car;
        }
    }

    class Director {
        public void constructSportsCar(Builder builder) {
            builder.reset();
            builder.setSeats("2 seats");
            builder.setEngine("V8 engine");
        }
    }

    public class builder {
        public static void main(String[] args) {
            Director director = new Director();
            CarBuilder builder = new CarBuilder();
            director.constructSportsCar(builder);
            Car car = builder.getProduct();
            car.show();
        }
    }
}
```

Prototype

Description: Creates new objects by **copying (cloning) an existing object**, rather than creating from scratch.

- **Why it is used:** When object creation is expensive or complex, cloning an existing object is faster and simpler. It avoids the overhead of re-initialization.
- **Where it is commonly used:** Game development (duplicating characters or levels), graphic editors (duplicating shapes), document templates.

Code Example (JAVA)

```
interface Prototype {
    Prototype clone();
}

class Shape implements Prototype {
    public int x, y;
    public String color;

    public Shape() {
    }
}
```

```
public Shape(Shape target) {
    if (target != null) {
        this.x = target.x;
        this.y = target.y;
        this.color = target.color;
    }
}

public Prototype clone() {
    return new Shape(this);
}
}

public class prototype {
    public static void main(String[] args) {
        Shape circle = new Shape();
        circle.x = 10;
        circle.y = 20;
        circle.color = "Red";

        Shape clonedCircle = (Shape) circle.clone();
        System.out.println("Cloned Circle Color: " + clonedCircle.color);
    }
}
```

2. Structural Design Patterns

Overview

Structural patterns deal with **how classes and objects are composed** to form larger structures. They simplify the structure by identifying relationships between entities and help ensure that when one part of a system changes, the entire structure does not need to change.

Adapter

Description: Acts as a **bridge between two incompatible interfaces**. It wraps an existing class with a new interface so that it becomes compatible with the client's expectations.

- **Why it is used:** To allow classes with incompatible interfaces to work together without modifying their source code.
 - **Where it is commonly used:** Integrating third-party libraries, legacy code integration, data format conversion (e.g., XML to JSON).
-

Code Example (JAVA)

```
interface Target {
    void request();
}
```

```
class Adaptee {
    public void specificRequest() {
        System.out.println("Specific request");
    }
}

class Adapter implements Target {
    private Adaptee adaptee = new Adaptee();

    public void request() {
        adaptee.specificRequest();
    }
}

public class adapter {
    public static void main(String[] args) {
        Target target = new Adapter();
        target.request();
    }
}
```

Bridge

Description: Decouples an abstraction from its implementation so that the two can vary independently.

- **Why it is used:** To avoid a permanent binding between an abstraction and its implementation and to allow both to be extended independently.
- **Where it is commonly used:** Device driver systems, rendering engines (OpenGL vs. DirectX), platform-independent UI frameworks.

Code Example (JAVA)

```
interface Device {
    void turnOn();
}

class TV implements Device {
    public void turnOn() {
        System.out.println("TV on");
    }
}

class Remote {
    protected Device device;

    public Remote(Device device) {
        this.device = device;
    }
}
```

```
        public void togglePower() {
            device.turnOn();
        }
    }

    public class bridge {
        public static void main(String[] args) {
            Remote remote = new Remote(new TV());
            remote.togglePower();
        }
    }
}
```

Composite

Description: Composes objects into **tree structures to represent part-whole hierarchies**. It lets clients treat individual objects and compositions of objects uniformly.

- **Why it is used:** To simplify client code that deals with tree structures by treating individual items and groups in the same way.
- **Where it is commonly used:** File system structures (files and folders), UI component trees, organization hierarchies.

Code Example (JAVA)

```
import java.util.ArrayList;
import java.util.List;

interface Graphic {
    void draw();
}

class Dot implements Graphic {
    public void draw() {
        System.out.println("Dot");
    }
}

class CompoundGraphic implements Graphic {
    private List<Graphic> children = new ArrayList<>();

    public void add(Graphic g) {
        children.add(g);
    }

    public void draw() {
        for (Graphic g : children)
            g.draw();
    }
}
```

```
public class composite {
    public static void main(String[] args) {
        CompoundGraphic comp = new CompoundGraphic();
        comp.add(new Dot());
        comp.draw();
    }
}
```

Decorator

Description: Adds new behavior or responsibilities to an object dynamically, without altering its class. It wraps the original object and adds functionality on top.

- **Why it is used:** To extend object functionality at runtime without using inheritance, keeping classes flexible and following the Open/Closed Principle.
- **Where it is commonly used:** I/O streams in Java, middleware in web frameworks, text formatting systems.

Code Example (JAVA)

```
interface DataSource {
    void writeData(String data);
}

class FileDataSource implements DataSource {
    public void writeData(String data) {
        System.out.println("Write: " + data);
    }
}

class EncryptionDecorator implements DataSource {
    private DataSource wrappee;

    public EncryptionDecorator(DataSource source) {
        this.wrappee = source;
    }

    public void writeData(String data) {
        wrappee.writeData("Encrypted(" + data + ")");
    }
}

public class decorator {
    public static void main(String[] args) {
        DataSource d = new EncryptionDecorator(new FileDataSource());
        d.writeData("Hello");
    }
}
```

Facade

Description: Provides a **simplified, unified interface** to a complex subsystem or set of interfaces.

- **Why it is used:** To hide complexity from the client. The client interacts with a simple interface instead of dealing with multiple complex components.
 - **Where it is commonly used:** API wrappers, library interfaces, startup/shutdown systems in complex applications.
-

Code Example (JAVA)

```
class VideoFile {
}

class CodecFactory {
}

class BitrateReader {
}

class AudioMixer {
}

class VideoConverter {
    public void convertVideo(String fileName, String format) {
        System.out.println("Converting " + fileName);
    }
}

public class facade {
    public static void main(String[] args) {
        VideoConverter converter = new VideoConverter();
        converter.convertVideo("file.mp4", "ogg");
    }
}
```

Flyweight

Description: Uses **sharing to efficiently support a large number of fine-grained objects**. It separates intrinsic (shared) state from extrinsic (unique) state.

- **Why it is used:** To reduce memory usage when a large number of similar objects need to be created.
 - **Where it is commonly used:** Text editors (reusing character objects), game engines (reusing tile or bullet objects), rendering systems.
-

Code Example (JAVA)

```
import java.util.HashMap;
import java.util.Map;

class TreeType {
    private String name, color;

    public TreeType(String name, String color) {
        this.name = name;
        this.color = color;
    }

    public void draw(int x, int y) {
        System.out.println("Tree " + name + " (" + color + ") at " + x +
            ", " + y);
    }
}

class TreeFactory {
    static Map<String, TreeType> treeTypes = new HashMap<>();

    public static TreeType getTreeType(String name, String color) {
        if (!treeTypes.containsKey(name))
            treeTypes.put(name, new TreeType(name, color));
        return treeTypes.get(name);
    }
}

public class flyweight {
    public static void main(String[] args) {
        TreeType type = TreeFactory.getTreeType("Oak", "Green");
        type.draw(10, 20);
    }
}
```

Proxy

Description: Provides a **substitute or placeholder** for another object to control access to it.

- **Why it is used:** To add a layer of control over the original object — for access control, lazy initialization, logging, or caching — without changing the original object.
- **Where it is commonly used:** Virtual proxies (lazy loading images), security proxies (access control), remote proxies (network calls).

Code Example (JAVA)

```
interface Subject {
    void request();
}
```

```
class RealSubject implements Subject {
    public void request() {
        System.out.println("RealSubject request");
    }
}

class Proxy implements Subject {
    private RealSubject realSubject;

    public void request() {
        if (realSubject == null)
            realSubject = new RealSubject();
        realSubject.request();
    }
}

public class proxy {
    public static void main(String[] args) {
        Subject p = new Proxy();
        p.request();
    }
}
```

3. Behavioral Design Patterns

Overview

Behavioral patterns deal with **how objects interact and communicate** with each other. They focus on the assignment of responsibilities between objects and define clear communication patterns to make the system more flexible in carrying out behavior.

Chain of Responsibility

Description: Passes a request along a **chain of handlers**, where each handler decides to process the request or pass it to the next handler in the chain.

- **Why it is used:** To decouple the sender of a request from its receivers and give multiple objects the chance to handle the request.
 - **Where it is commonly used:** Middleware pipelines (HTTP request handling), event handling systems, logging with multiple levels.
-

Code Example (JAVA)

```
public class chain_of_responsibility {  
    public static void main(String[] args) {  
        System.out.println("chain_of_responsibility");  
    }  
}
```

Command

Description: **Encapsulates a request as an object**, allowing you to parameterize clients with different requests, queue operations, and support undoable actions.

- **Why it is used:** To decouple the object that sends a request from the object that executes it, enabling flexible command management.
 - **Where it is commonly used:** Undo/redo systems, task schedulers, GUI buttons and menu actions, transactional operations.
-

Code Example (JAVA)

```
public class command {  
    public static void main(String[] args) {  
        System.out.println("command");  
    }  
}
```

Iterator

Description: Provides a way to **sequentially access elements** of a collection without exposing its underlying representation.

- **Why it is used:** To traverse different types of collections in a uniform way without depending on their internal structure.
 - **Where it is commonly used:** Iterating over lists, trees, graphs, and custom data structures in any programming language.
-

Code Example (JAVA)

```
public class iterator { public static void main(String[] args) {  
    System.out.println("iterator"); } }
```

Mediator

Description: Defines an object that **encapsulates how a set of objects interact**. It promotes loose coupling by preventing objects from referring to each other directly.

- **Why it is used:** To reduce the chaotic dependencies between many objects communicating directly. All communication goes through the mediator.
 - **Where it is commonly used:** Chat room systems, air traffic control systems, UI dialog form coordination.
-

Code Example (JAVA)

```
public class mediator {  
    public static void main(String[] args) {  
        System.out.println("mediator");  
    }  
}
```

Memento

Description: Captures and externalizes an object's **internal state** so it can be restored later, without violating encapsulation.

- **Why it is used:** To implement undo/redo functionality by saving snapshots of an object's state.
 - **Where it is commonly used:** Text editors (undo history), game save states, transactional systems.
-

Code Example (JAVA)

```
public class memento {  
    public static void main(String[] args) {  
        System.out.println("memento");  
    }  
}
```

Observer

Description: Defines a **one-to-many dependency** between objects so that when one object changes state, all its dependents are notified and updated automatically.

- **Why it is used:** To implement event-driven systems where multiple parts of an application need to react to changes in a shared object.
 - **Where it is commonly used:** Event listeners, pub/sub systems, real-time dashboards, MVC architecture (model notifying views).
-

Code Example (JAVA)

```
public class observer {  
    public static void main(String[] args) {
```

```
        System.out.println("observer");  
    }  
}
```

State

Description: Allows an object to **alter its behavior when its internal state changes**. The object will appear to change its class.

- **Why it is used:** To replace complex conditional statements (`if/switch`) that depend on the object's state with dedicated state classes.
- **Where it is commonly used:** Traffic light systems, vending machines, order status workflows, game character states.

Code Example (JAVA)

```
interface State {  
    void doAction(Context context);  
}  
  
class StartState implements State {  
    public void doAction(Context context) {  
        System.out.println("Player is in start state");  
        context.setState(this);  
    }  
}  
  
class StopState implements State {  
    public void doAction(Context context) {  
        System.out.println("Player is in stop state");  
        context.setState(this);  
    }  
}  
  
class Context {  
    private State state;  
  
    public void setState(State state) {  
        this.state = state;  
    }  
  
    public State getState() {  
        return state;  
    }  
}  
  
public class state {  
    public static void main(String[] args) {  
        Context context = new Context();  
        State start = new StartState();  
    }  
}
```

```
        start.doAction(context);  
    }  
}
```

Strategy

Description: Defines a **family of algorithms**, encapsulates each one, and makes them interchangeable. The strategy lets the algorithm vary independently from clients that use it.

- **Why it is used:** To select behavior at runtime without changing the client that uses it. Promotes the Open/Closed Principle.
- **Where it is commonly used:** Sorting algorithms (choose bubble sort vs. quicksort at runtime), payment processing systems, data compression strategies.

Code Example (JAVA)

```
interface Strategy {  
    int execute(int a, int b);  
}  
  
class Add implements Strategy {  
    public int execute(int a, int b) {  
        return a + b;  
    }  
}  
  
class Subtract implements Strategy {  
    public int execute(int a, int b) {  
        return a - b;  
    }  
}  
  
class Context {  
    private Strategy strategy;  
  
    public void setStrategy(Strategy strategy) {  
        this.strategy = strategy;  
    }  
  
    public int executeStrategy(int a, int b) {  
        return strategy.execute(a, b);  
    }  
}  
  
public class strategy {  
    public static void main(String[] args) {  
        Context context = new Context();  
        context.setStrategy(new Add());  
        System.out.println("Add: " + context.executeStrategy(3, 4));  
    }  
}
```

```
    }  
}
```

Template Method

Description: Defines the **skeleton of an algorithm** in a base class, but lets subclasses override specific steps without changing the algorithm's overall structure.

- **Why it is used:** To avoid code duplication when multiple classes share the same algorithm structure but differ in specific steps.
- **Where it is commonly used:** Data parsing pipelines, report generation systems, game AI turn sequences.

Code Example (JAVA)

```
public class template_method {  
    public static void main(String[] args) {  
        System.out.println("template_method");  
    }  
}
```

Visitor

Description: Lets you **add new operations to existing object structures** without modifying the objects. A visitor object carries the operation to be performed.

- **Why it is used:** To separate an algorithm from the object structure it operates on, making it easy to add new operations without touching existing classes.
- **Where it is commonly used:** Compilers (AST traversal), document export systems (export to PDF, HTML, XML), tax/discount calculation systems.

Summary Table

Category	Focus	Goal
Creational	Object creation	Control how objects are instantiated
Structural	Object composition	Build flexible and efficient structures
Behavioral	Object interaction	Define clear communication between objects

Tip for Beginners: Don't try to memorize all patterns at once. Start with the most commonly used ones — **Singleton**, **Factory Method**, **Observer**, **Strategy**, and **Decorator** — and learn the rest as you encounter real-world problems they solve.

Code Example (JAVA)

```
public class visitor {  
    public static void main(String[] args) {  
        System.out.println("visitor");  
    }  
}
```